

Project Status

Team Members

1. Rishikesh Kasireddy- ES23BTECH11033
2. Soham Mashetty- ES23BTECH11036

Overview

We have been given a fully working single cycle RISC-V simulator and high level objectives were to

1. Add a 5-stage pipeline with selectable modes.
 - a. Mode 1: Single-cycle baseline (existing)
 - b. Mode 2: Pipelined (no hazard handling)
 - c. Mode 3: Pipelined with hazard detection (stalls)
 - d. Mode 4: Pipelined with hazard detection + forwarding
 - e. Mode 5: With static branch prediction
 - f. Mode 6: With dynamic 1-bit prediction
2. Provide detailed cycle-accurate simulation of pipeline behavior.
3. Develop a Performance Dashboard to automatically measure and compare metrics (total cycles, CPI, stall count, forwarding events, mispredictions)

What have we done so far

Till date we have successfully implemented the simulator till mode 3 i.e Hazard detection with only stalling

The Major changes in the project can be put into the following categories. We will elaborate on what exactly we have added in the following sections.

1. We reorganized the project so that **all execution modes** and features are controlled through a single file: config.h
2. Implemented a 5 stage pipeline which will be executed in reverse order
3. Implemented a hazard detection unit which will be invoked in the ID stage

Top-level architecture & config changes

We made a centralized config file config.h. This sets the control signals based on the mode chosen in the VM.

- This lets us turn modes on and off without changing the datapath
- Let us add new modes later by just changing config flags and making control logic to read from the config file.
- Lets different pipeline behaviours run under same VM binary

Reverse (Back to front pipeline)

In our main file RV5s.cpp the main driver RV5SVM::Step() explicitly does:

```
void RV5SVM::Step() {
    current_delta_.old_pc = program_counter_;
    current_delta_.ifid = if_id_;
    current_delta_.idex = id_ex_;
    current_delta_.exmem = ex_mem_;
    current_delta_.memwb = mem_wb_;
    current_delta_.cycle_s = cycle_s_;
    current_delta_.instructions_retired = instructions_retired_;
    current_delta_.register_changes.clear();
    current_delta_.memory_changes.clear();

    //Reverse Pipeline stages
    stageWB();
    stageMEM();
    stageEX();
    stageID();
    stageIF();
}
```

This is the **reverse pipeline execution order**: WB -> MEM -> EX -> ID -> IF.

Each stage has its own function where it would do the defined actions. IF fetches instructions and updates PC, ID stage decodes instructions etc and sets the appropriate control signals. In the function step, each stage is called in reverse order from WB to IF. And in the run function we call the step function in a loop until the PC >= Program size.

Why reverse order??

Problems in forward pipeline

- Writes occur in WB during the first half of the cycle and registers are read in ID during the second half
- forward implementation required storing both "previous" and "current" register/file states and careful timing of writes vs reads.

Reverse approach

- By executing WB first, register writes are committed before ID executes its reads for the same cycle. This naturally enforces a "write-first, then read" semantics
- It also eliminates the need to double-buffer register file values or to keep prior-cycle copies for most cases.

Resulting simplification:

- We no longer needed to store both current and previous register values in the forward pipeline just to avoid read-after-write races.
- Pipeline register updates use local out objects, so there is no accidental next-state pollution (each stage computes its next state from the *current* pipeline registers, then commits).

How we Implemented Reverse order

Every stage constructs a stage-specific out struct (e.g., IFID out{}, IDEX out{}, MEMWB out{}), populates fields derived from the current pipeline registers and state, then assigns `if_id_ = out / id_ex_ = out / mem_wb_ = out`.

```
void RV5SVM::stageIF() {
    IFID out{};
    if (program_counter_ < program_size_) {
        out.instr = memory_controller_.ReadWord(program_counter_);
        out.pc = program_counter_;
        out.valid = true;
        // If we must stall IF/ID, do not advance PC; else advance sequentially
        if (!stall_if_id_) {
            UpdateProgramCounter(4);
        }
    }
    // Apply one-cycle IF flush if requested (control hazard from ID)
    if (flush_if_once_) {
        out = {}; // discard fetched instruction this cycle
        flush_if_once_ = false;
    }
    // Commit IF/ID only if not stalling; otherwise freeze previous IF/ID
    if (!stall_if_id_) {
        if_id_ = out;
    }
}
```

Hazard Detection (stall only) - mechanism & exact stall policy

Core algorithm :

- Decode RS1/RS2 from the ID instruction opcode.
- Determine whether ID uses rs1/rs2 (helper `id_uses_rs1`, `id_uses_rs2`).
- For each producer stage (EX, MEM, WB) compute if producer.rd collides with `id.rs1/rs2` and the producer writes.
- Set stall cycles depending on where the dependency is:
 - EX → 2 stalls
 - MEM → 1 stall
 - WB → 0 stalls

```

void RV5SVM::stageID() {
    // Hazard detection (stall-only): compute stalls and IF flush request
    stall_if_id_ = false;
    if (is_stall_mode()) {
        // If we are in the middle of a stall burst, continue stalling
        if (stall_counter_ > 0) {
            stall_if_id_ = true;
            id_ex_ = {}; // bubble
            stall_counter_--;
            return;
        }
        // Fresh computation from current pipeline state
        HazardDecision h = hazard_.Compute(if_id_, id_ex_, ex_mem_, mem_wb_);
        if (h.flush_if) {
            flush_if_once_ = true;
        }
        if (h.stall_cycles > 0) {
            stall_counter_ = h.stall_cycles - 1; // we consume one stall this cycle
            stall_if_id_ = true;
            id_ex_ = {}; // bubble
            return;
        }
    }
}

```

- StageID() uses a stall counter to create bubbles: when stall counter > 0, it inserts a bubble into id ex and decrements the counter.
- IF/ID commit is frozen when a stall is in effect.

```

    if (!stall_if_id_) {
        if_id_ = out;
    }
}

```

Undo-Redo Functionality

Undo and Redo are defined as two stacks with each node being a struct called. The VM captures a current_delta_ per cycle with:

- register_changes vector storing old and new GPR values (used by Undo()).
- memory_changes to restore memory modifications.
- old_pc, new_pc, to store values of the program counter
- lfid, idex, exmem, memwb for the stages

Undo restores prior state by popping deltas and pushing to the redo stack and Redo restores the future state by popping the delta and pushing it back to the undo stack.

Following is the undo functionality

```
void RV5SVM::Undo() {
    if (undo_stack_.empty()) {
        std::cout << "VM_NO_MORE_UNDO" << std::endl;
        return;
    }
    StepDelta5 last = undo_stack_.top();
    undo_stack_.pop();

    redo_stack_.push(last);
    std::cout << "VM_UNDO_COMPLETED" << std::endl;
}
```

Following is the redo functionality

```
void RV5SVM::Redo() {
    if (redo_stack_.empty()) {
        std::cout << "VM_NO_MORE_REDO" << std::endl;
        return;
    }
    StepDelta5 next = redo_stack_.top();
    redo_stack_.pop();

    undo_stack_.push(next);
    std::cout << "VM_REDO_COMPLETED" << std::endl;
}
```

Following is the struct definition of each node for the stacks.

```
You, last week | 1 author (You)
struct StepDelta5
{
    uint64_t old_pc;
    uint64_t new_pc;
    std::vector<RegisterChange5> register_changes;
    std::vector<MemoryChange5> memory_changes;
    IFID ifid;
    IDEX idex;
    EXMEM exmem;
    MEMWB memwb;
    uint64_t cycle_s;
    uint64_t instructions_retired;
};
```

Expecting challenges in upcoming stages

Stage 4 - Forwarding

- In Mode-3, we always stall for dependencies. With forwarding, we only stall in load use cases.
- Forwarding gives the EX stage the **latest available value** needed for computation, instead of waiting for Write-Back to complete.

Where forwarded values can come from

- Data may come from three locations:
 1. The **EX/MEM** pipeline register (newest ALU result)
 2. The **MEM/WB** pipeline register (memory result or ALU result)
 3. The **register file** (if already written back)
- We always prefer the most recent data.

Changes required in EX stage

- Before doing the ALU operation:
 - Check if RS1 or RS2 matches the **destination register** in EX/MEM or MEM/WB stage
 - If it matches and the instruction writes a result we then **forward that value**
 - Otherwise use RS1/RS2 value read in ID stage
- A helper function like ResolveOperand will be added to handle this logic.

Load-use hazard remains

- Forwarding **cannot** fix when a value is still inside the MEMstage

Ex: LD x3, 0(x1)
 ADD x4, x3, x5

We must install one stall in such cases

Going forward, the next major focus will be Mode-4, where forwarding will be implemented to reduce unnecessary stalls and improve throughput. After that, branch prediction mechanisms will be introduced with branch predictors and branch history tables. We are also planning to include cache mechanisms and 2 bit predictors if time permits.